

CAPÍTULO 18

Grafos

Objetivos

Con el estudio de este capítulo usted podrá:

- Distinguir entre relaciones jerárquicas y otras relaciones.
- Definir un grafo e identificar sus componentes.
- Conocer estructuras de datos para representar un grafo.
- Conocer las operaciones básicas que se aplican sobre grafos.
- Encontrar los caminos que puede haber entre dos nodos de un grafo.
- Realizar en C++ la representación de los grafos y las operaciones básicas.
- Calcular el camino mínimo desde un vértice al resto de los vértices.
- Determinar si hay camino entre cualquier par de vértices de un grafo.
- Implementar los algoritmos más importantes sobre grafos.
- Representar con un grafo ciertas relaciones entre objetos y aplicar los algoritmos que determinan su conectividad.
- Saber cuándo se aplica un algoritmo de expansión de coste mínimo y cuándo un camino de coste mínimo.

Contenido

- 18.1. Conceptos y definiciones.
- 18.2. Representación de los grafos.
- 18.3. Listas de adyacencia.
- 18.4. Recorrido de un grafo.
- 18.5. Conexiones en un grafo.
- 18.6. Matriz de caminos. Cierre transitivo.
- 18.7. Ordenación topológica.
- 18.8. Matriz de caminos: Algoritmo de Warshall.
- 18.9. Caminos más cortos con un solo origen: algoritmo de Dijkstra.
- 18.10. Todos los caminos mínimos: Algoritmo de Floyd.

- 18.11. Árbol de expansión de coste mínimo.

LECTURAS RECOMENDADAS.

RESUMEN.
EJERCICIOS.
PROBLEMAS.
ANEXOS C y D en página web del libro (lecturas recomendadas para profundizar en el tema).

Conceptos clave

- Árbol de expansión.
- Camino.
- Caminos mínimos.
- Conexión y componente conexa.
- Factor de peso.
- Lista de adyacencia.
- Matriz de adyacencia.
- Ordenación topológica.
- Relación jerárquica.
- Vértice y arco.

INTRODUCCIÓN

Este capítulo introduce al lector a conceptos matemáticos importantes denominados grafos que tienen aplicaciones en campos tan diversos como sociología, química, geografía, ingeniería eléctrica e industrial, etc. Los grafos se estudian como estructuras de datos o tipos abstractos de datos. Este capítulo estudia las definiciones relativas a los grafos y la representación de los grafos en la memoria del ordenador. El capítulo investiga dos formas tradicionales de implementación de grafo, matriz de adyacencia y listas de adyacencia. También se estudian operaciones importantes y algoritmos de grafos que son significativos en informática.

Existen numerosos problemas que se pueden modelar en términos de grafos. Ejemplo de ello es la planificación de las tareas que completan un proyecto, encontrar las rutas de menor longitud entre dos puntos geográficos, calcular el camino más rápido en un transporte, determinar el flujo máximo que puede llegar desde una *fuentes* a, por ejemplo, una urbanización.

La resolución de estos problemas requiere examinar todos los nodos y las aristas del grafo que representa al problema. Los algoritmos imponen implícitamente un orden en estas visitas: el nodo más próximo o las aristas más cortas, y así sucesivamente; no todos los algoritmos requieren un orden concreto en el recorrido del grafo.

Según lo anterior, se estudian en este capítulo el concepto de ordenación topológica, los problemas del camino más corto, junto con el concepto de árbol de expansión de coste mínimo. Estos algoritmos han sido desarrollado por grandes investigadores y en su honor se conocen por su nombre los algoritmos de Dijkstra, Warshall, Prim, Kruscal, Ford-Fulkerson ...

18.1. CONCEPTOS Y DEFINICIONES

Un grafo G agrupa *entes* físico o conceptuales y las relaciones entre ellos. Por tanto, un grafo está formado por un conjunto de vértices o *nodos* V , que representan a los *entes*, y un conjunto de arcos A , que representan las relaciones entre vértices. Se representa con el par $G = (V, A)$. La Figura 18.1 muestra un grafo formado por los vértices $V = \{1, 4, 5, 7, 9\}$ y el conjunto de arcos $A = \{(1, 4), (4, 1), (5, 1), (1, 5), (7, 9), (9, 7), (7, 5), (5, 7), (4, 9), (9, 4)\}$

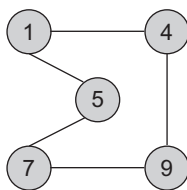


Figura 18.1. Grafo no dirigido.

Un arco o arista representa una *relación* entre dos nodos. Esta relación, al estar formada por dos nodos, se representa por (u, v) siendo u, v el par de nodos. El grafo es *no dirigido* si los arcos están formados por pares de nodos no ordenados, no apuntados; se representa con un segmento uniendo los nodos, $u - v$. El grafo de la Figura 18.1 es no dirigido.

Un grafo es *dirigido*, también denominado *digrafo*, si los pares de nodos que forman los arcos son ordenados; se representan con una flecha que indica la dirección de la relación,

$u \rightarrow v$. El grafo de la Figura 18.2 que consta de los vértices, $V = \{C, D, E, F, H\}$, y de los arcos $A = \{(C, D), (D, F), (E, H), (H, E), (E, C)\}$ forman el grafo dirigido $G = \{V, A\}$

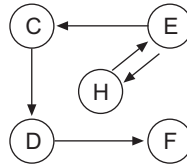


Figura 18.2. Grafo dirigido.

Dado el arco (u, v) de un grafo, se dice que los vértices u y v son adyacentes. Si el grafo es dirigido, el vértice u es adyacente a v , y v es adyacente de u .

En los modelos realizados con grafos, a veces, una relación entre dos nodos tiene asociada una magnitud, denominada factor de peso, en cuyo caso se dice que es un grafo valorado. Por ejemplo, los pueblos que forman una comarca junto a la relación entre un par de pueblos de estar unidos por un camino: esta relación tiene asociado el factor de peso, que es la distancia en kilómetros. La Figura 18.3 muestra un grafo valorado en el que cada arco tiene asociado un peso que es la longitud entre dos nodos.

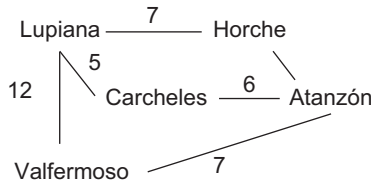


Figura 18.3. Grafo no dirigido valorado.

Definición

Un grafo permite modelar relaciones arbitrarias entre objetos. Un grafo $G = (V, A)$ es un par formado por un conjunto de vértices o nodos, V , y un conjunto de arcos o aristas, A . Cada arco es el par (u, w) , siendo u, w dos vértices relacionados.

18.1.1. Grado de entrada, grado de salida de un nodo

El **grado** es una cualidad que se refiere a los nodos de un grafo. En un grafo no dirigido el grado de un nodo v , $\text{grado}(v)$, es el número de arcos que contiene a v . En un grafo dirigido se distingue entre grado de entrada y grado de salida; *grado de entrada* de un nodo v , $\text{gradient}(v)$, es el número de arcos que llegan a v ; grado de salida de v , $\text{gradsal}(v)$, es el número de arcos que salen de v .

Así, en el grafo no dirigido de la Figura 18.3, $\text{grado}(\text{Lupiana}) = 3$. En el grafo dirigido de la Figura 18.2, $\text{gradient}(D) = 1$ y el $\text{gradsal}(D) = 1$.

18.1.2. Camino

Un camino P de longitud n desde el vértice v_0 a v_n en un grafo G , es la secuencia de $n + 1$ vértices $v_0, v_1, v_2, \dots, v_n$ tal que $(v_i, v_{i+1}) \in A(\text{arcos})$ para $0 \leq i < n$. Matemáticamente el camino se representa por $P = (v_0, v_1, v_2, \dots, v_n)$.

En la Figura 18.4 se pueden encontrar más de un camino; por ejemplo, $P_1 = (4, 6, 9, 7)$ es un camino de longitud 3. Otro de los caminos es $P_2 = (10, 11)$, que tiene de longitud 1. Por tanto, se puede afirmar que la longitud del camino es el número de arcos que lo forman.

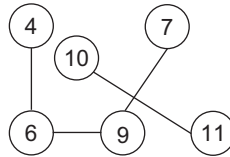


Figura 18.4. Grafo no dirigido de 5 vértices.

Definición

La longitud de un camino en un grafo no valorado es el número de arcos en el camino. En un grafo valorado, la longitud del camino con pesos es la suma de los pesos de los arcos en el camino.

En el grafo valorado de la Figura 18.3, el camino (Lupiana, Valfermoso, Atanzón) tiene de longitud $12 + 7 = 19$.

En algunos grafos se dan arcos desde un vértice a sí mismo, (v, v) ; entonces el camino $v \rightarrow v$ es un bucle. Normalmente, en los grafos no hay nodos relacionados con sí mismo, no es frecuente encontrarse grafos con bucles.

Un camino $P = (v_0, v_1, v_2, \dots, v_n)$ es simple si todos los nodos que forman el camino son distintos, pudiendo ser iguales los extremos del camino v_0, v_n . En el grafo de la Figura 18.4 el camino P_1 y P_2 son caminos simples.

En un grafo dirigido, un ciclo es un camino simple cerrado. Por tanto, un ciclo empieza y termina en el mismo nodo, $v_0 = v_n$, y, además, debe tener más de un arco. Un grafo dirigido sin ciclos (acíclico) se acostumbra a denominar mediante la abreviatura GDA (Grafo Dirigido Acíclico). La Figura 18.5 muestra un grafo dirigido en el que los vértices (A, E, B, F, A) forman un ciclo de longitud 4. En general, un ciclo de longitud k se denomina k -ciclo.

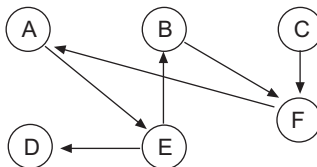


Figura 18.5. Grafo dirigido con ciclos.

Un grafo no dirigido es **conexo** si existe un camino entre cualquier par de nodos que forman el grafo. En el caso de un grafo dirigido con esta propiedad se dice que es **fuertemente conexo**. Además, un **grafo completo** es aquél que tiene un arco para cualquier par de vértices.

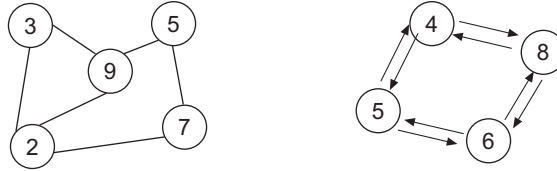


Figura 18.6. a) Grafo conexo. b) Grafo fuertemente conexo.

18.1.3. Tipo Abstracto de Datos Grafo

Un grafo consta de un conjunto de objetos o elementos junto a sus relaciones. Es preciso definir las operaciones básicas para construir la estructura y, en general, modificar sus elementos. En definitiva, especificar el *tipo abstracto de datos grafo*.

Ahora se definen operaciones básicas, a partir de las cuales se construye el grafo. Su realización depende de la representación elegida (matriz de adyacencia, o listas de adyacencia).

arista(u, v). Añade el arco o arista(u, v) al grafo.

aristaPeso(u, v, w). Para un grafo valorado, añade el arco(u, v) al grafo y el coste del arco, w .

borraArco(u, v). Elimina del grafo el arco(u, v).

adyacente(u, v). Operación que devuelve *cierto* si los vértices u, v forman un arco.

nuevoVertice(u). Añade el vértice u al grafo G .

borraVértice(u). Elimina el vértice u del grafo G .

18.2. REPRESENTACIÓN DE LOS GRAFOS

Para trabajar con los grafos y aplicar algoritmos que permitan encontrar propiedades entre los nodos hay que pensar en su representación. Cómo representar un grafo en memoria interna, qué tipos o estructuras de datos se deben utilizar para considerar los nodos y los arcos.

Una primera simplificación que se hace es considerar a los vértices o nodos como números consecutivos, empezando por el vértice 0. Hay que tener en cuenta que se tiene que representar un número (finito) de vértices y de arcos que unen dos vértices. Se puede elegir una **representación secuencial**, mediante un array bidimensional, conocida como **matriz de adyacencia**; o bien, una **representación dinámica**, mediante una estructura multienlazada, denominada **listas de adyacencia**. La elección de una representación u otra depende del tipo de grafo y de las operaciones que se vayan a realizar sobre los vértices y arcos. Para un **grafo denso** (tiene la mayoría de los arcos posibles) lo mejor es utilizar una **matriz de adyacencia**. Para un **grafo disperso** (tiene, relativamente, pocos arcos) se suele utilizar **listas de adyacencia** que se ajustan al número de arcos.

18.2.1. Matriz de adyacencia

La característica más importante de un grafo, que distingue a un grafo de otro, es el conjunto de pares de vértices que están *relacionados*, o que son adyacentes. Por ello, la forma más sen-

cilla de representar un grafo es mediante una matriz, de tantas filas / columnas como nodos, que permite modelar fácilmente esa cualidad.

Sea $G = (V, A)$ un grafo de n nodos, siendo $V = \{v_0, v_1, \dots, v_{n-1}\}$ el conjunto de nodos, y $A = \{(v_i, v_j)\}$ el conjunto de arcos. Los nodos se pueden representar por números consecutivos de 0 a $n - 1$. La representación de los arcos se hace con una matriz A de $n \times n$ elementos, denominada matriz de adyacencia, tal que todo elemento a_{ij} puede tomar los valores:

$$a_{ij} = \begin{cases} 1 & \text{si hay un arco } (v_i, v_j) \\ 0 & \text{si no hay arco } (v_i, v_j) \end{cases}$$

La Figura 18.7 representa un grafo dirigido, suponiendo que el orden de los vértices es el de la secuencia $\{D, F, K, L, R\}$, entonces la matriz de adyacencia:

$$A = \begin{vmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{vmatrix}$$

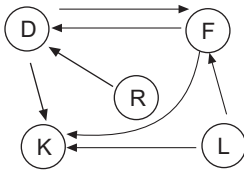


Figura 18.7. Grafo dirigido con los vértices $\{D, F, K, L, R\}$.

El grafo de la Figura 18.8 es no dirigido, está formado por 5 vértices. La matriz de adyacencia:

$$A = \begin{vmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \end{vmatrix}$$

puede observar, es una matriz simétrica. En los grafos no dirigidos la matriz de adyacencia siempre es simétrica ya que las relaciones entre vértices no son ordenadas: si v_i está relacionado con v_j , entonces v_j está relacionado con v_i .

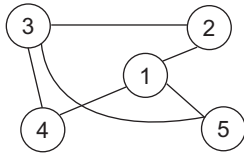


Figura 18.8. Grafo no dirigido con 5 vértices.

Los grafos que modelan problemas en los que un arco tiene asociado una magnitud, un *factor de peso*, también se representan mediante una matriz de tantas filas / columnas como nodos. Ahora un elemento cualquiera, a_{ij} representa el coste o *factor de peso del arco* (v_i, v_j). Un arco que no existe se puede representar con un valor imposible, por ejemplo, *infinito*; también, se puede representar con el valor 0, dependiendo de que para el grafo el 0 no pueda ser un factor de peso significativo de un arco. A esta matriz también se la denomina *matriz valorada*.

El grafo valorado de la Figura 18.9 se corresponde con un *grafo dirigido con factor de peso*. Si se supone que los vértices se numeran en el orden de $V = \{\text{Alicante, Barcelona, Cartagena, Murcia, Reus}\}$, la matriz de pesos P en la que se representa con 0 la no existencia de arco:

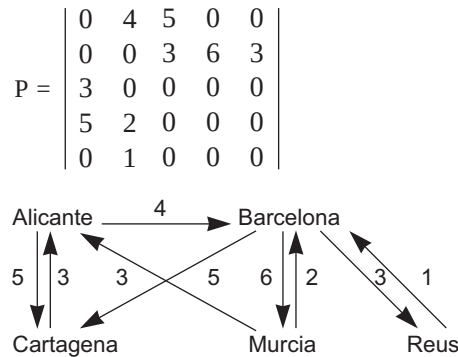


Figura 18.9. Grafo dirigido con factor de peso.

A considerar

La matriz de adyacencia representa los arcos, las relaciones entre un par de nodos de un grafo. Es una matriz de unos y ceros, que indican que dos vértices son adyacentes o no. En un grafo valorado, cada elemento representa el peso de la arista, y por ello se la denomina *matriz de pesos*.

18.2.2. Matriz de adyacencia: class GrafoMatriz

A todos los nodos del grafo se les da un nombre (string), y se le asigna un número, a partir de 0, en la matriz de adyacencia. La clase `Vertice` representa un nodo del grafo. Sus tres constructores sobrecargados son los encargados de inicializar los dos atributos protegidos `nombre` y `numVertice`. El primero de ellos lo hace por defecto; el segundo inicializa el nombre y pone como número de vértice -1; el tercero inicializa el número de vértice y el nombre. El resto de los métodos de la clase `Vertice` realizan las funciones de acceder o modificar cada uno de los atributos. La función miembro `igual()`, decide si el nombre de un vértice que recibe como parámetro coincide con el del objeto actual.

```
class Vertice
{
protected:
    string nombre;
    int numVertice;
```

```

public:
    Vertice () {}
    Vertice(string x)
    {
        // inicializa el nombre y pone el número de vértice e -1
        nombre = x;
        numVertice = -1;
    }
    Vertice(string x, int n)
    {
        // inicializa el nombre y el número de vértice
        nombre = x;
        numVertice = n;
    }
    string OnomVertice()
    {
        // retorna el nombre del vértice
        return nombre;
    }
    void PnomVertice(string nom)
    {
        // cambia el nombre del vértice
        nombre = nom;
    }
    bool igual(Vertice n)
    {
        // decide entre la igualdad de nombres
        return nombre == n.nombre;
    }
    void PnumVertice(int n)
    {
        // cambia el número del vértice
        numVertice = n;
    }
    int OnumVertice()
    {
        // retorna el número del vértice
        return numVertice;
    }
};

```

La clase **GrafoMatriz** contiene como atributos protegidos su tamaño (número máximo de vértices), el número de vértices actual, la matriz de adyacencia de dimensión variable y un array dimensionable que almacena los distintos *Vértices*. Además, contiene todos los métodos públicos necesarios para acceder o modificar cada uno de sus atributos. Posteriormente, se añadirán métodos que implementan operaciones comunes.

```

class GrafoMatriz
{
    protected:
        int maxVerts;        // máximo numero de vértices
        int numVerts;        // número de vértices actual
        Vertice * verts;     // array de vértices
        int ** matAd;        // matriz de adyacencia
    public:
        // métodos públicos de la clase GrafoMatriz
    private:
        // métodos privados de la clase GrafoMariz
};

```

El constructor sin argumentos crea la matriz de adyacencia para un máximo de vértices preestablecido que en este caso es 1; el otro constructor tiene un argumento con el máximo

número de vértices. Para dimensionar la matriz de adyacencia se necesita declarar como tipo predefinido un puntero a entero. La implementación de los métodos públicos es:

```
typedef int * pint; // para el dimensionamiento de la matriz

GrafoMatriz::GrafoMatriz(int mx)
{
    maxVerts = mx;
    verts = new Vertice[mx] ;           // vector de vértices
    matAd = new pint[mx];               // vector de punteros
    numVerts = 0;
    for (int i = 0; i < mx; i++)
        matAd[i] = new int [mx];       // matriz de adyacencia
}

GrafoMatriz::GrafoMatriz()
{
    maxVerts = 1 ;
    GrafoMatriz(maxVerts);
}
```

Las funciones miembro públicas encargadas de obtener el número de vértices o de cambiar el número de vértices de la clase GrafoMatriz son:

```
int OnumeroDeVertices(){return numVerts;}
void PnumeroDeVertices(int n){numVerts = n;}
```

Añadir un vértice

La operación nuevoVertice() recibe el nombre de un vértice del grafo, comprueba si el nombre recibido se encuentra en la lista de vértices incrementando en caso negativo el número de vértices y asignándolo a la lista. La comprobación de si el nombre está en lista de vértices es realizado por el método numVertice().

```
void GrafoMatriz::nuevoVertice (string nom)
{
    bool esta = numVertice(nom) >= 0;
    if (!esta)
    {
        Vertice v = Vertice(nom, numVerts);
        verts[numVerts++] = v; // se asigna a la lista.
        // No se comprueba que sobrepase el máximo
    }
}
```

numVertice() es un método público de la clase, busca el vértice en el array, retornando -1 si no lo encuentra:

```
int GrafoMatriz::numVertice(string v)
{
    int i;
    bool encontrado = false;
    for ( i = 0; (i < numVerts) && !encontrado;)
```

```

    {
        encontrado = verts[i].igual(v);
        if (!encontrado) i++ ;
    }
    return (i < numVerts) ? i : -1 ;
}

```

Añadir un arco

El método sobrecargado y público `nuevoArco()` recibe el nombre de los dos vértices que forman el arco y busca en el array, el número de vértice asignado a cada uno de ellos marcando la matriz de adyacencia.

```

void GrafoMatriz::nuevoArco(string a, string b)
{
    int va, vb;
    va = numVertice(a);
    vb = numVertice(b);
    if (va < 0 || vb < 0) throw "Vértice no existe";
    matAd[va][vb] = 1;
}

```

Otra versión sobrecargada del método público recibe, directamente, los números de los vértices que forman el arco y marca la posición de la matriz.

```

void GrafoMatriz::nuevoArco(int va, int vb)
{
    if (va < 0 || vb < 0 || va > numVerts || vb > numVerts)
        throw "Vértice no existe";
    matAd[va][vb] = 1;
}

```

Este método tiene un tercer argumento para *grafos valorados*, que representa el *factor de peso* del arco que se asigna a la matriz. Por ejemplo, en el caso anterior, la versión de la función miembro es:

```

void GrafoMatriz::nuevoArco(int va, int vb, int valor)
{
    if (va < 0 || vb < 0 || va > numVerts || vb > numVerts)
        throw "Vértice no existe";
    matAd[va][vb] = valor;
}

```

Para realizar la entrada completa del grafo en memoria, primero se asignan los vértices y, a continuación, sus arcos. El tiempo de esta operación depende de la densidad del grafo, si se considera un grafo denso el tiempo de ejecución es cuadrático, $O(n^2)$, siendo n el número de nodos.

Adyacente

Determina si dos vértices, v_1 y v_2 , forman un arco. Sencillamente, si el elemento de la matriz de adyacencia es 1. Se escriben dos versiones de la función miembro pública `adyacente()`. La primera tiene los nombres de los vértices como parámetro y la segunda los números de vértice.

```

bool GrafoMatriz::adyacente(string a, string b)
{
    int va, vb;
    va = numVertice(a);
    vb = numVertice(b);
    if (va < 0 || vb < 0) throw "Vértice no existe";
    return matAd[va][vb] == 1;
}

bool GrafoMatriz::adyacente(int va, int vb)
{
    if (va < 0 || vb < 0 || va >= numVerts || vb >= numVerts)
        throw "Vértice no existe";
    return matAd[va][vb] == 1;
}

```

Valor de la matriz de adyacencia

El método público sobrecargado `Ovalor()` se encarga de retornar el valor almacenado en la matriz de adyacencia para los vértices que recibe como parámetro, identificados con el nombre del vértice o con el número asignado. Es muy similar al método `adyacente()`, excepto que en este caso se retorna el propio contenido de la matriz, por lo que puede servir para grafos valorados. Hay que tener en cuenta que esta función es muy importante para el tratamiento de los algoritmos de grafos, ya que retorna el valor de las distintas entradas de la matriz de adyacencia o de pesos para grafos no valorados o valorados respectivamente.

```

int GrafoMatriz::Ovalor(string a, string b)
{
    int va, vb;
    va = numVertice(a);
    vb = numVertice(b);
    if (va < 0 || vb < 0) throw "Vértice no existe";
    return matAd[va][vb];
}

int GrafoMatriz::Ovalor( int va, int vb)
{
    if (va < 0 || vb < 0 || va >= numVerts || vb >= numVerts)
        throw "Vértice no existe";
    return matAd[va][vb];
}

```

Modificar el valor de la matriz de adyacencia

El método público `Pvalor()` modifica el valor de la posición de la matriz de adyacencia que recibe como parámetro, o bien mediante el nombre o bien mediante el número del vértice. La función miembro es interesante para usarla tanto en grafos valorados como no valorados ya que permite modificar una entrada de la matriz de pesos o de adyacencia.

```

void GrafoMatriz::Pvalor(int va, int vb, int v)
{
    if (va < 0 || vb < 0 || va >= numVerts || vb >= numVerts)

```

```

        throw "Vértice no existe";
    matAd[va][vb] = v;
}

void GrafoMatriz::Pvalor( char *a, char *b, int v)
{
    int va, vb;
    va = numVertice(a);
    vb = numVertice(b);
    if (va < 0 || vb < 0) throw "Vértice no existe";

    matAd[va][vb] = v;
}

```

Vértice asociado a un número entero

El método público `Overtice()` recibe como parámetro una identificación del vértice y retorna todos los atributos del vértice si está almacenado en el grafo. La importancia de esta función miembro radica en permitir obtener toda la información asociada a un vértice almacenada en el grafo, para su posterior consulta. Sólo se codifica la versión que recibe como parámetro el número entero que identifica al vértice, y se deja al lector la versión que recibe como parámetro el nombre del vértice.

```

Vertice Overtice(int va)
{
    if (va < 0 || va >= numVerts)
        throw "Vértice no existe";
    else return verts[va]
}

```

Resulta interesante el método público `Pvertice()` que permite modificar la información de un vértice, para el caso de que el vértice se identifique como un número entero.

```

void Pvertice( int va, Vertice vert)
{
    if (va < 0 || va >= numVerts)
        throw "Vértice no existe";
    else verts[va] = vert;
}

```

18.3. LISTAS DE ADYACENCIA

La representación de un grafo con matriz de adyacencia no es eficiente cuando el grafo es poco denso, (disperso), es decir tiene **pocos arcos** y, por tanto, la matriz de adyacencia tiene muchos ceros. Para grafos dispersos la matriz de adyacencia ocupa el mismo espacio que si el grafo tuviera muchos arcos (grafo denso). Cuando esto ocurre, se elige la representación del grafo mediante listas enlazadas, denominadas **listas de adyacencia**.

Las listas de adyacencia son una **estructura multienlazada formada por una tabla directorio donde cada elemento de la tabla representa un vértice del grafo, del que emerge una lista enlazada con todos sus vértices adyacentes. Esto es, cada lista representa a los arcos con vértice origen el del nodo de la lista directorio, por eso se llama lista de adyacencia.**

La Figura 19.10 representa a un grafo dirigido, la representación mediante listas de adyacencia se encuentra en la Figura 19.11. Si por ejemplo se analiza el vértice 5, es adyacente a los vértices 1, 2 y 4; por ello su lista de adyacencia consta de tres nodos, cada uno con el vértice destino que forma el arco. También se puede observar que el vértice 4 no es origen de ningún arco, como consecuencia la lista de adyacencia está vacía.

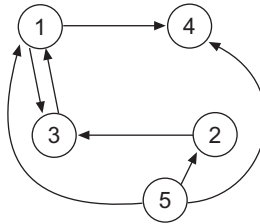


Figura 18.10. Grafo dirigido.

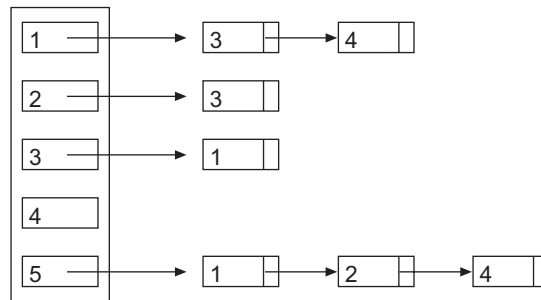


Figura 18.11. Listas de adyacencia del grafo 19.10.

Nota

La implementación de un grafo con listas de adyacencia puede consultarse en la lectura recomendada del Anexo C del capítulo.

18.4. RECORRIDO DE UN GRAFO

En general, *recorrer una estructura* consiste en visitar (procesar) cada uno de los nodos a partir de uno dado; así se ha realizado el recorrido de un árbol en, por ejemplo, *preorden* partiendo del nodo raíz. De igual forma, *recorrer un grafo* consiste en **visitar todos los vértices alcanzables a partir de uno dado**. Muchos de los problemas que se plantean con los grafos exigen examinar las aristas de que consta y procesar los vértices del grafo.

Sea V el conjunto de vértices del grafo, los algoritmos de *recorrido* de grafos parten de un nodo, v , y mantienen un conjunto de nodos *marcados* (*procesados*), w , que inicialmente es el nodo o vértice de partida, v . En cada *pasada* del algoritmo se retira un nodo, w , del conjunto w , se procesa y para cada una de las aristas que tengan como origen el vértice w , (w, u) , su